

CS229 Lecture notes

Andrew Ng

Part XI

Principal components analysis

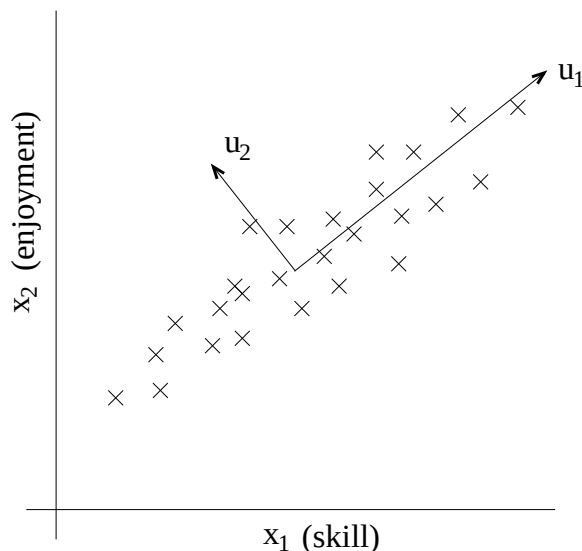
In our discussion of factor analysis, we gave a way to model data $x \in \mathbb{R}^n$ as “approximately” lying in some k -dimension subspace, where $k \ll n$. Specifically, we imagined that each point $x^{(i)}$ was created by first generating some $z^{(i)}$ lying in the k -dimension affine space $\{\Lambda z + \mu; z \in \mathbb{R}^k\}$, and then adding Ψ -covariance noise. Factor analysis is based on a probabilistic model, and parameter estimation used the iterative EM algorithm.

In this set of notes, we will develop a method, Principal Components Analysis (PCA), that also tries to identify the subspace in which the data approximately lies. However, PCA will do so more directly, and will require only an eigenvector calculation (easily done with the `eig` function in Matlab), and does not need to resort to EM.

Suppose we are given a dataset $\{x^{(i)}; i = 1, \dots, m\}$ of attributes of m different types of automobiles, such as their maximum speed, turn radius, and so on. Let $x^{(i)} \in \mathbb{R}^n$ for each i ($n \ll m$). But unknown to us, two different attributes—some x_i and x_j —respectively give a car’s maximum speed measured in miles per hour, and the maximum speed measured in kilometers per hour. These two attributes are therefore almost linearly dependent, up to only small differences introduced by rounding off to the nearest mph or kph. Thus, the data really lies approximately on an $n - 1$ dimensional subspace. How can we automatically detect, and perhaps remove, this redundancy?

For a less contrived example, consider a dataset resulting from a survey of pilots for radio-controlled helicopters, where $x_1^{(i)}$ is a measure of the piloting skill of pilot i , and $x_2^{(i)}$ captures how much he/she enjoys flying. Because RC helicopters are very difficult to fly, only the most committed students, ones that truly enjoy flying, become good pilots. So, the two attributes x_1 and x_2 are strongly correlated. Indeed, we might posit that that the

data actually lies along some diagonal axis (the u_1 direction) capturing the intrinsic piloting “karma” of a person, with only a small amount of noise lying off this axis. (See figure.) How can we automatically compute this u_1 direction?



We will shortly develop the PCA algorithm. But prior to running PCA per se, typically we first pre-process the data to normalize its mean and variance, as follows:

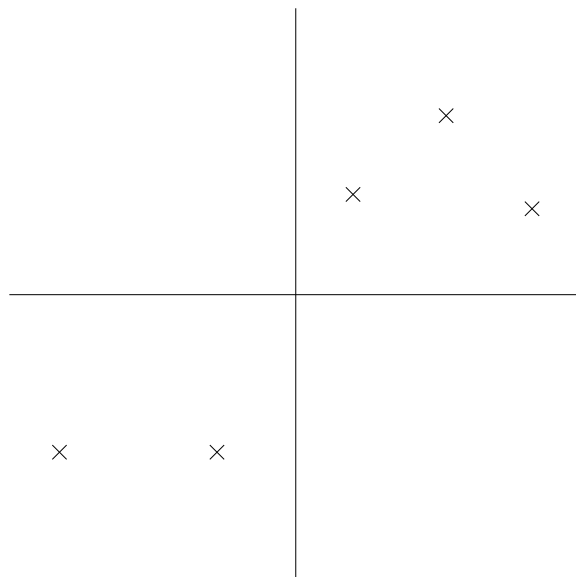
1. Let $\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$.
2. Replace each $x^{(i)}$ with $x^{(i)} - \mu$.
3. Let $\sigma_j^2 = \frac{1}{m} \sum_i (x_j^{(i)})^2$
4. Replace each $x_j^{(i)}$ with $x_j^{(i)} / \sigma_j$.

Steps (1-2) zero out the mean of the data, and may be omitted for data known to have zero mean (for instance, time series corresponding to speech or other acoustic signals). Steps (3-4) rescale each coordinate to have unit variance, which ensures that different attributes are all treated on the same “scale.” For instance, if x_1 was cars’ maximum speed in mph (taking values in the high tens or low hundreds) and x_2 were the number of seats (taking values around 2-4), then this renormalization rescales the different attributes to make them more comparable. Steps (3-4) may be omitted if we had apriori knowledge that the different attributes are all on the same scale. One

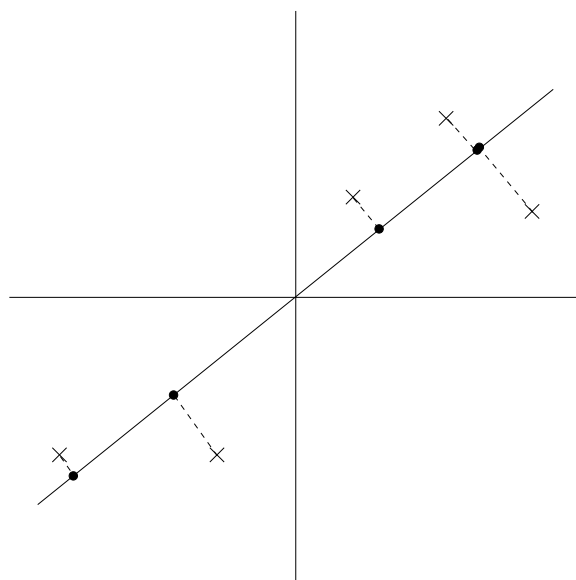
example of this is if each data point represented a grayscale image, and each $x_j^{(i)}$ took a value in $\{0, 1, \dots, 255\}$ corresponding to the intensity value of pixel j in image i .

Now, having carried out the normalization, how do we compute the “major axis of variation” u —that is, the direction on which the data approximately lies? One way to pose this problem is as finding the unit vector u so that when the data is projected onto the direction corresponding to u , the variance of the projected data is maximized. Intuitively, the data starts off with some amount of variance/information in it. We would like to choose a direction u so that if we were to approximate the data as lying in the direction/subspace corresponding to u , as much as possible of this variance is still retained.

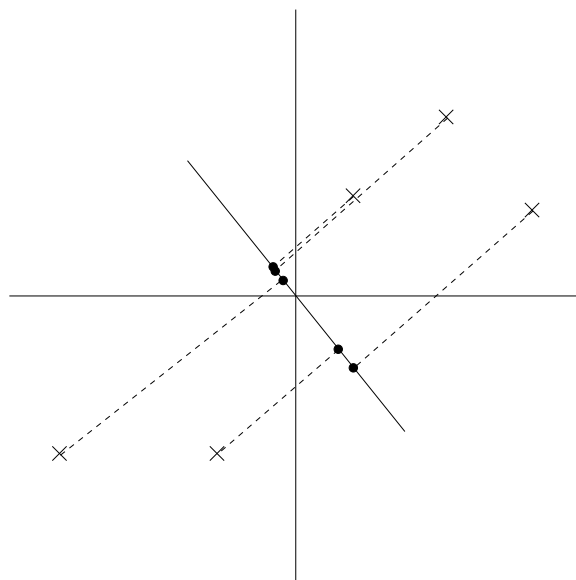
Consider the following dataset, on which we have already carried out the normalization steps:



Now, suppose we pick u to correspond to the direction shown in the figure below. The circles denote the projections of the original data onto this line.



We see that the projected data still has a fairly large variance, and the points tend to be far from zero. In contrast, suppose had instead picked the following direction:



Here, the projections have a significantly smaller variance, and are much closer to the origin.

We would like to automatically select the direction u corresponding to the first of the two figures shown above. To formalize this, note that given a

unit vector u and a point x , the length of the projection of x onto u is given by $x^T u$. I.e., if $x^{(i)}$ is a point in our dataset (one of the crosses in the plot), then its projection onto u (the corresponding circle in the figure) is distance $x^{(i)T} u$ from the origin. Hence, to maximize the variance of the projections, we would like to choose a unit-length u so as to maximize:

$$\begin{aligned} \frac{1}{m} \sum_{i=1}^m (x^{(i)T} u)^2 &= \frac{1}{m} \sum_{i=1}^m u^T x^{(i)} x^{(i)T} u \\ &= u^T \left(\frac{1}{m} \sum_{i=1}^m x^{(i)} x^{(i)T} \right) u. \end{aligned}$$

We easily recognize that the maximizing this subject to $\|u\|_2 = 1$ gives the principal eigenvector of $\Sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} x^{(i)T}$, which is just the empirical covariance matrix of the data (assuming it has zero mean).¹

To summarize, we have found that if we wish to find a 1-dimensional subspace with which to approximate the data, we should choose u to be the principal eigenvector of Σ . More generally, if we wish to project our data into a k -dimensional subspace ($k < n$), we should choose $u_1, \dots, u_k$ to be the top k eigenvectors of Σ . The u_i 's now form a new, orthogonal basis for the data.²

Then, to represent $x^{(i)}$ in this basis, we need only compute the corresponding vector

$$y^{(i)} = \begin{bmatrix} u_1^T x^{(i)} \\ u_2^T x^{(i)} \\ \vdots \\ u_k^T x^{(i)} \end{bmatrix} \in \mathbb{R}^k.$$

Thus, whereas $x^{(i)} \in \mathbb{R}^n$, the vector $y^{(i)}$ now gives a lower, k -dimensional, approximation/representation for $x^{(i)}$. PCA is therefore also referred to as a **dimensionality reduction** algorithm. The vectors $u_1, \dots, u_k$ are called the first k **principal components** of the data.

Remark. Although we have shown it formally only for the case of $k = 1$, using well-known properties of eigenvectors it is straightforward to show that

¹If you haven't seen this before, try using the method of Lagrange multipliers to maximize $u^T \Sigma u$ subject to that $u^T u = 1$. You should be able to show that $\Sigma u = \lambda u$, for some λ , which implies u is an eigenvector of Σ , with eigenvalue λ .

²Because Σ is symmetric, the u_i 's will (or always can be chosen to be) orthogonal to each other.

of all possible orthogonal bases $u_1, \dots, u_k$, the one that we have chosen maximizes $\sum_i \|y^{(i)}\|_2^2$. Thus, our choice of a basis preserves as much variability as possible in the original data.

In problem set 4, you will see that PCA can also be derived by picking the basis that minimizes the approximation error arising from projecting the data onto the k -dimensional subspace spanned by them.

PCA has many applications; we will close our discussion with a few examples. First, compression—representing $x^{(i)}$'s with lower dimension $y^{(i)}$'s—is an obvious application. If we reduce high dimensional data to $k = 2$ or 3 dimensions, then we can also plot the $y^{(i)}$'s to visualize the data. For instance, if we were to reduce our automobiles data to 2 dimensions, then we can plot it (one point in our plot would correspond to one car type, say) to see what cars are similar to each other and what groups of cars may cluster together.

Another standard application is to preprocess a dataset to reduce its dimension before running a supervised learning algorithm with the $x^{(i)}$'s as inputs. Apart from computational benefits, reducing the data's dimension can also reduce the complexity of the hypothesis class considered and help avoid overfitting (e.g., linear classifiers over lower dimensional input spaces will have smaller VC dimension).

Lastly, as in our RC pilot example, we can also view PCA as a noise reduction algorithm. In our example it estimates the intrinsic “piloting karma” from the noisy measures of piloting skill and enjoyment. In class, we also saw the application of this idea to face images, resulting in **eigenfaces** method. Here, each point $x^{(i)} \in \mathbb{R}^{100 \times 100}$ was a 10000 dimensional vector, with each coordinate corresponding to a pixel intensity value in a 100x100 image of a face. Using PCA, we represent each image $x^{(i)}$ with a much lower-dimensional $y^{(i)}$. In doing so, we hope that the principal components we found retain the interesting, systematic variations between faces that capture what a person really looks like, but not the “noise” in the images introduced by minor lighting variations, slightly different imaging conditions, and so on. We then measure distances between faces i and j by working in the reduced dimension, and computing $\|y^{(i)} - y^{(j)}\|_2$. This resulted in a surprisingly good face-matching and retrieval algorithm.

CS229 Lecture notes

Andrew Ng

Part XII

Independent Components Analysis

Our next topic is Independent Components Analysis (ICA). Similar to PCA, this will find a new basis in which to represent our data. However, the goal is very different.

As a motivating example, consider the “cocktail party problem.” Here, n speakers are speaking simultaneously at a party, and any microphone placed in the room records only an overlapping combination of the n speakers’ voices. But let’s say we have n different microphones placed in the room, and because each microphone is a different distance from each of the speakers, it records a different combination of the speakers’ voices. Using these microphone recordings, can we separate out the original n speakers’ speech signals?

To formalize this problem, we imagine that there is some data $s \in \mathbb{R}^n$ that is generated via n independent sources. What we observe is

$$x = As,$$

where A is an unknown square matrix called the **mixing matrix**. Repeated observations gives us a dataset $\{x^{(i)}; i = 1, \dots, m\}$, and our goal is to recover the sources $s^{(i)}$ that had generated our data ($x^{(i)} = As^{(i)}$).

In our cocktail party problem, $s^{(i)}$ is an n -dimensional vector, and $s_j^{(i)}$ is the sound that speaker j was uttering at time i . Also, $x^{(i)}$ is an n -dimensional vector, and $x_j^{(i)}$ is the acoustic reading recorded by microphone j at time i .

Let $W = A^{-1}$ be the **unmixing matrix**. Our goal is to find W , so that given our microphone recordings $x^{(i)}$, we can recover the sources by computing $s^{(i)} = Wx^{(i)}$. For notational convenience, we also let w_i^T denote

the i -th row of W , so that

$$W = \begin{bmatrix} - & w_1^T & - \\ & \vdots & \\ - & w_n^T & - \end{bmatrix}.$$

Thus, $w_i \in \mathbb{R}^n$, and the j -th source can be recovered by computing $s_j^{(i)} = w_j^T x^{(i)}$.

1 ICA ambiguities

To what degree can $W = A^{-1}$ be recovered? If we have no prior knowledge about the sources and the mixing matrix, it is not hard to see that there are some inherent ambiguities in A that are impossible to recover, given only the $x^{(i)}$'s.

Specifically, let P be any n -by- n permutation matrix. This means that each row and each column of P has exactly one “1.” Here’re some examples of permutation matrices:

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}; \quad P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; \quad P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

If z is a vector, then Pz is another vector that’s contains a permuted version of z ’s coordinates. Given only the $x^{(i)}$ ’s, there will be no way to distinguish between W and PW . Specifically, the permutation of the original sources is ambiguous, which should be no surprise. Fortunately, this does not matter for most applications.

Further, there is no way to recover the correct scaling of the w_i ’s. For instance, if A were replaced with $2A$, and every $s^{(i)}$ were replaced with $(0.5)s^{(i)}$, then our observed $x^{(i)} = 2A \cdot (0.5)s^{(i)}$ would still be the same. More broadly, if a single column of A were scaled by a factor of α , and the corresponding source were scaled by a factor of $1/\alpha$, then there is again no way, given only the $x^{(i)}$ ’s to determine that this had happened. Thus, we cannot recover the “correct” scaling of the sources. However, for the applications that we are concerned with—including the cocktail party problem—this ambiguity also does not matter. Specifically, scaling a speaker’s speech signal $s_j^{(i)}$ by some positive factor α affects only the volume of that speaker’s speech. Also, sign changes do not matter, and $s_j^{(i)}$ and $-s_j^{(i)}$ sound identical when played on a speaker. Thus, if the w_i found by an algorithm is scaled by any non-zero real

number, the corresponding recovered source $s_i = w_i^T x$ will be scaled by the same factor; but this usually does not matter. (These comments also apply to ICA for the brain/MEG data that we talked about in class.)

Are these the only sources of ambiguity in ICA? It turns out that they are, so long as the sources s_i are *non-Gaussian*. To see what the difficulty is with Gaussian data, consider an example in which $n = 2$, and $s \sim \mathcal{N}(0, I)$. Here, I is the 2×2 identity matrix. Note that the contours of the density of the standard normal distribution $\mathcal{N}(0, I)$ are circles centered on the origin, and the density is rotationally symmetric.

Now, suppose we observe some $x = As$, where A is our mixing matrix. The distribution of x will also be Gaussian, with zero mean and covariance $E[xx^T] = E[Ass^T A^T] = AA^T$. Now, let R be an arbitrary orthogonal (less formally, a rotation/reflection) matrix, so that $RR^T = R^T R = I$, and let $A' = AR$. Then if the data had been mixed according to A' instead of A , we would have instead observed $x' = A's$. The distribution of x' is also Gaussian, with zero mean and covariance $E[x'(x')^T] = E[A'ss^T (A')^T] = E[ARss^T (AR)^T] = ARR^T A^T = AA^T$. Hence, whether the mixing matrix is A or A' , we would observe data from a $\mathcal{N}(0, AA^T)$ distribution. Thus, there is no way to tell if the sources were mixed using A and A' . So, there is an arbitrary rotational component in the mixing matrix that cannot be determined from the data, and we cannot recover the original sources.

Our argument above was based on the fact that the multivariate standard normal distribution is rotationally symmetric. Despite the bleak picture that this paints for ICA on Gaussian data, it turns out that, so long as the data is *not* Gaussian, it is possible, given enough data, to recover the n independent sources.

2 Densities and linear transformations

Before moving on to derive the ICA algorithm proper, we first digress briefly to talk about the effect of linear transformations on densities.

Suppose we have a random variable s drawn according to some density $p_s(s)$. For simplicity, let us say for now that $s \in \mathbb{R}$ is a real number. Now, let the random variable x be defined according to $x = As$ (here, $x \in \mathbb{R}$, $A \in \mathbb{R}$). Let p_x be the density of x . What is p_x ?

Let $W = A^{-1}$. To calculate the “probability” of a particular value of x , it is tempting to compute $s = Wx$, then evaluate p_s at that point, and conclude that “ $p_x(x) = p_s(Wx)$.” However, *this is incorrect*. For example, let $s \sim \text{Uniform}[0, 1]$, so that s ’s density is $p_s(s) = 1\{0 \leq s \leq 1\}$. Now, let

$A = 2$, so that $x = 2s$. Clearly, x is distributed uniformly in the interval $[0, 2]$. Thus, its density is given by $p_x(x) = (0.5)1\{0 \leq x \leq 2\}$. This does not equal $p_s(Wx)$, where $W = 0.5 = A^{-1}$. Instead, the correct formula is $p_x(x) = p_s(Wx)|W|$.

More generally, if s is a vector-valued distribution with density p_s , and $x = As$ for a square, invertible matrix A , then the density of x is given by

$$p_x(x) = p_s(Wx) \cdot |W|,$$

where $W = A^{-1}$.

Remark. If you've seen the result that A maps $[0, 1]^n$ to a set of volume $|A|$, then here's another way to remember the formula for p_x given above, that also generalizes our previous 1-dimensional example. Specifically, let $A \in \mathbb{R}^{n \times n}$ be given, and let $W = A^{-1}$ as usual. Also let $C_1 = [0, 1]^n$ be the n -dimensional hypercube, and define $C_2 = \{As : s \in C_1\} \subseteq \mathbb{R}^n$ to be the image of C_1 under the mapping given by A . Then it is a standard result in linear algebra (and, indeed, one of the ways of defining determinants) that the volume of C_2 is given by $|A|$. Now, suppose s is uniformly distributed in $[0, 1]^n$, so its density is $p_s(s) = 1\{s \in C_1\}$. Then clearly x will be uniformly distributed in C_2 . Its density is therefore found to be $p_x(x) = 1\{x \in C_2\}/\text{vol}(C_2)$ (since it must integrate over C_2 to 1). But using the fact that the determinant of the inverse of a matrix is just the inverse of the determinant, we have $1/\text{vol}(C_2) = 1/|A| = |A^{-1}| = |W|$. Thus, $p_x(x) = 1\{x \in C_2\}|W| = 1\{Wx \in C_1\}|W| = p_s(Wx)|W|$.

3 ICA algorithm

We are now ready to derive an ICA algorithm. The algorithm we describe is due to Bell and Sejnowski, and the interpretation we give will be of their algorithm as a method for maximum likelihood estimation. (This is different from their original interpretation, which involved a complicated idea called the infomax principal, that is no longer necessary in the derivation given the modern understanding of ICA.)

We suppose that the distribution of each source s_i is given by a density p_{s_i} , and that the joint distribution of the sources s is given by

$$p(s) = \prod_{i=1}^n p_{s_i}(s_i).$$

Note that by modeling the joint distribution as a product of the marginal, we capture the assumption that the sources are independent. Using our

formulas from the previous section, this implies the following density on $x = As = W^{-1}s$:

$$p(x) = \prod_{i=1}^n p_s(w_i^T x) \cdot |W|.$$

All that remains is to specify a density for the individual sources p_s .

Recall that, given a real-valued random variable z , its cumulative distribution function (cdf) F is defined by $F(z_0) = P(z \leq z_0) = \int_{-\infty}^{z_0} p_z(z) dz$. Also, the density of z can be found from the cdf by taking its derivative: $p_z(z) = F'(z)$.

Thus, to specify a density for the s_i 's, all we need to do is to specify some cdf for it. A cdf has to be a monotonic function that increases from zero to one. Following our previous discussion, we cannot choose the cdf to be the cdf of the Gaussian, as ICA doesn't work on Gaussian data. What we'll choose instead for the cdf, as a reasonable "default" function that slowly increases from 0 to 1, is the sigmoid function $g(s) = 1/(1 + e^{-s})$. Hence, $p_s(s) = g'(s)$.¹

The square matrix W is the parameter in our model. Given a training set $\{x^{(i)}; i = 1, \dots, m\}$, the log likelihood is given by

$$\ell(W) = \sum_{i=1}^m \left(\sum_{j=1}^n \log g'(w_j^T x^{(i)}) + \log |W| \right).$$

We would like to maximize this in terms W . By taking derivatives and using the fact (from the first set of notes) that $\nabla_W |W| = |W|(W^{-1})^T$, we easily derive a stochastic gradient ascent learning rule. For a training example $x^{(i)}$, the update rule is:

$$W := W + \alpha \left(\begin{bmatrix} 1 - 2g(w_1^T x^{(i)}) \\ 1 - 2g(w_2^T x^{(i)}) \\ \vdots \\ 1 - 2g(w_n^T x^{(i)}) \end{bmatrix} x^{(i)T} + (W^T)^{-1} \right),$$

¹If you have prior knowledge that the sources' densities take a certain form, then it is a good idea to substitute that in here. But in the absence of such knowledge, the sigmoid function can be thought of as a reasonable default that seems to work well for many problems. Also, the presentation here assumes that either the data $x^{(i)}$ has been preprocessed to have zero mean, or that it can naturally be expected to have zero mean (such as acoustic signals). This is necessary because our assumption that $p_s(s) = g'(s)$ implies $E[s] = 0$ (the derivative of the logistic function is a symmetric function, and hence gives a density corresponding to a random variable with zero mean), which implies $E[x] = E[As] = 0$.

where α is the learning rate.

After the algorithm converges, we then compute $s^{(i)} = Wx^{(i)}$ to recover the original sources.

Remark. When writing down the likelihood of the data, we implicitly assumed that the $x^{(i)}$'s were independent of each other (for different values of i ; note this issue is different from whether the different coordinates of $x^{(i)}$ are independent), so that the likelihood of the training set was given by $\prod_i p(x^{(i)}; W)$. This assumption is clearly incorrect for speech data and other time series where the $x^{(i)}$'s are dependent, but it can be shown that having correlated training examples will not hurt the performance of the algorithm if we have sufficient data. But, for problems where successive training examples are correlated, when implementing stochastic gradient ascent, it also sometimes helps accelerate convergence if we visit training examples in a randomly permuted order. (I.e., run stochastic gradient ascent on a randomly shuffled copy of the training set.)

CS229 Lecture notes

Andrew Ng

Part XIII

Reinforcement Learning and Control

We now begin our study of reinforcement learning and adaptive control.

In supervised learning, we saw algorithms that tried to make their outputs mimic the labels y given in the training set. In that setting, the labels gave an unambiguous “right answer” for each of the inputs x . In contrast, for many sequential decision making and control problems, it is very difficult to provide this type of explicit supervision to a learning algorithm. For example, if we have just built a four-legged robot and are trying to program it to walk, then initially we have no idea what the “correct” actions to take are to make it walk, and so do not know how to provide explicit supervision for a learning algorithm to try to mimic.

In the reinforcement learning framework, we will instead provide our algorithms only a reward function, which indicates to the learning agent when it is doing well, and when it is doing poorly. In the four-legged walking example, the reward function might give the robot positive rewards for moving forwards, and negative rewards for either moving backwards or falling over. It will then be the learning algorithm’s job to figure out how to choose actions over time so as to obtain large rewards.

Reinforcement learning has been successful in applications as diverse as autonomous helicopter flight, robot legged locomotion, cell-phone network routing, marketing strategy selection, factory control, and efficient web-page indexing. Our study of reinforcement learning will begin with a definition of the **Markov decision processes (MDP)**, which provides the formalism in which RL problems are usually posed.

1 Markov decision processes

A Markov decision process is a tuple $(S, A, \{P_{sa}\}, \gamma, R)$, where:

- S is a set of **states**. (For example, in autonomous helicopter flight, S might be the set of all possible positions and orientations of the helicopter.)
- A is a set of **actions**. (For example, the set of all possible directions in which you can push the helicopter's control sticks.)
- P_{sa} are the state transition probabilities. For each state $s \in S$ and action $a \in A$, P_{sa} is a distribution over the state space. We'll say more about this later, but briefly, P_{sa} gives the distribution over what states we will transition to if we take action a in state s .
- $\gamma \in [0, 1)$ is called the **discount factor**.
- $R : S \times A \mapsto \mathbb{R}$ is the **reward function**. (Rewards are sometimes also written as a function of a state S only, in which case we would have $R : S \mapsto \mathbb{R}$).

The dynamics of an MDP proceeds as follows: We start in some state s_0 , and get to choose some action $a_0 \in A$ to take in the MDP. As a result of our choice, the state of the MDP randomly transitions to some successor state s_1 , drawn according to $s_1 \sim P_{s_0 a_0}$. Then, we get to pick another action a_1 . As a result of this action, the state transitions again, now to some $s_2 \sim P_{s_1 a_1}$. We then pick a_2 , and so on... Pictorially, we can represent this process as follows:

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

Upon visiting the sequence of states $s_0, s_1, \dots$ with actions $a_0, a_1, \dots$, our total payoff is given by

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \dots$$

Or, when we are writing rewards as a function of the states only, this becomes

$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

For most of our development, we will use the simpler state-rewards $R(s)$, though the generalization to state-action rewards $R(s, a)$ offers no special difficulties.

Our goal in reinforcement learning is to choose actions over time so as to maximize the expected value of the total payoff:

$$\mathbb{E} [R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots]$$

Note that the reward at timestep t is **discounted** by a factor of γ^t . Thus, to make this expectation large, we would like to accrue positive rewards as soon as possible (and postpone negative rewards as long as possible). In economic applications where $R(\cdot)$ is the amount of money made, γ also has a natural interpretation in terms of the interest rate (where a dollar today is worth more than a dollar tomorrow).

A **policy** is any function $\pi : S \mapsto A$ mapping from the states to the actions. We say that we are **executing** some policy π if, whenever we are in state s , we take action $a = \pi(s)$. We also define the **value function** for a policy π according to

$$V^\pi(s) = \mathbb{E} [R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots \mid s_0 = s, \pi].$$

$V^\pi(s)$ is simply the expected sum of discounted rewards upon starting in state s , and taking actions according to π .¹

Given a fixed policy π , its value function V^π satisfies the **Bellman equations**:

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in S} P_{s\pi(s)}(s') V^\pi(s').$$

This says that the expected sum of discounted rewards $V^\pi(s)$ for starting in s consists of two terms: First, the **immediate reward** $R(s)$ that we get rightaway simply for starting in state s , and second, the expected sum of future discounted rewards. Examining the second term in more detail, we see that the summation term above can be rewritten $\mathbb{E}_{s' \sim P_{s\pi(s)}}[V^\pi(s')]$. This is the expected sum of discounted rewards for starting in state s' , where s' is distributed according $P_{s\pi(s)}$, which is the distribution over where we will end up after taking the first action $\pi(s)$ in the MDP from state s . Thus, the second term above gives the expected sum of discounted rewards obtained *after* the first step in the MDP.

Bellman's equations can be used to efficiently solve for V^π . Specifically, in a finite-state MDP ($|S| < \infty$), we can write down one such equation for $V^\pi(s)$ for every state s . This gives us a set of $|S|$ linear equations in $|S|$ variables (the unknown $V^\pi(s)$'s, one for each state), which can be efficiently solved for the $V^\pi(s)$'s.

¹This notation in which we condition on π isn't technically correct because π isn't a random variable, but this is quite standard in the literature.

We also define the **optimal value function** according to

$$V^*(s) = \max_{\pi} V^{\pi}(s). \quad (1)$$

In other words, this is the best possible expected sum of discounted rewards that can be attained using any policy. There is also a version of Bellman's equations for the optimal value function:

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s'). \quad (2)$$

The first term above is the immediate reward as before. The second term is the maximum over all actions a of the expected future sum of discounted rewards we'll get upon after action a . You should make sure you understand this equation and see why it makes sense.

We also define a policy $\pi^* : S \mapsto A$ as follows:

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P_{sa}(s') V^*(s'). \quad (3)$$

Note that $\pi^*(s)$ gives the action a that attains the maximum in the “max” in Equation (2).

It is a fact that for every state s and every policy π , we have

$$V^*(s) = V^{\pi^*}(s) \geq V^{\pi}(s).$$

The first equality says that the V^{π^*} , the value function for π^* , is equal to the optimal value function V^* for every state s . Further, the inequality above says that π^* 's value is at least as large as the value of any other policy. In other words, π^* as defined in Equation (3) is the optimal policy.

Note that π^* has the interesting property that it is the optimal policy for *all* states s . Specifically, it is not the case that if we were starting in some state s then there'd be some optimal policy for that state, and if we were starting in some other state s' then there'd be some other policy that's optimal policy for s' . Specifically, the same policy π^* attains the maximum in Equation (1) for *all* states s . This means that we can use the same policy π^* no matter what the initial state of our MDP is.

2 Value iteration and policy iteration

We now describe two efficient algorithms for solving finite-state MDPs. For now, we will consider only MDPs with finite state and action spaces ($|S| < \infty$, $|A| < \infty$).

The first algorithm, **value iteration**, is as follows:

1. For each state s , initialize $V(s) := 0$.
2. Repeat until convergence {

For every state, update $V(s) := R(s) + \max_{a \in A} \gamma \sum_{s'} P_{sa}(s')V(s')$.

This algorithm can be thought of as repeatedly trying to update the estimated value function using Bellman Equations (2).

There are two possible ways of performing the updates in the inner loop of the algorithm. In the first, we can first compute the new values for $V(s)$ for every state s , and then overwrite all the old values with the new values. This is called a **synchronous** update. In this case, the algorithm can be viewed as implementing a “Bellman backup operator” that takes a current estimate of the value function, and maps it to a new estimate. (See homework problem for details.) Alternatively, we can also perform **asynchronous** updates. Here, we would loop over the states (in some order), updating the values one at a time.

Under either synchronous or asynchronous updates, it can be shown that value iteration will cause V to converge to V^* . Having found V^* , we can then use Equation (3) to find the optimal policy.

Apart from value iteration, there is a second standard algorithm for finding an optimal policy for an MDP. The **policy iteration** algorithm proceeds as follows:

1. Initialize π randomly.
2. Repeat until convergence {
 - (a) Let $V := V^\pi$.
 - (b) For each state s , let $\pi(s) := \arg \max_{a \in A} \sum_{s'} P_{sa}(s')V(s')$.

Thus, the inner-loop repeatedly computes the value function for the current policy, and then updates the policy using the current value function. (The policy π found in step (b) is also called the policy that is **greedy with respect to V** .) Note that step (a) can be done via solving Bellman’s equations as described earlier, which in the case of a fixed policy, is just a set of $|S|$ linear equations in $|S|$ variables.

After at most a finite number of iterations of this algorithm, V will converge to V^* , and π will converge to π^* .

Both value iteration and policy iteration are standard algorithms for solving MDPs, and there isn't currently universal agreement over which algorithm is better. For small MDPs, policy iteration is often very fast and converges with very few iterations. However, for MDPs with large state spaces, solving for V^π explicitly would involve solving a large system of linear equations, and could be difficult. In these problems, value iteration may be preferred. For this reason, in practice value iteration seems to be used more often than policy iteration.

3 Learning a model for an MDP

So far, we have discussed MDPs and algorithms for MDPs assuming that the state transition probabilities and rewards are known. In many realistic problems, we are not given state transition probabilities and rewards explicitly, but must instead estimate them from data. (Usually, S , A and γ are known.)

For example, suppose that, for the inverted pendulum problem (see problem set 4), we had a number of trials in the MDP, that proceeded as follows:

$$\begin{array}{ccccccc} s_0^{(1)} & \xrightarrow{a_0^{(1)}} & s_1^{(1)} & \xrightarrow{a_1^{(1)}} & s_2^{(1)} & \xrightarrow{a_2^{(1)}} & s_3^{(1)} \xrightarrow{a_3^{(1)}} \dots \\ s_0^{(2)} & \xrightarrow{a_0^{(2)}} & s_1^{(2)} & \xrightarrow{a_1^{(2)}} & s_2^{(2)} & \xrightarrow{a_2^{(2)}} & s_3^{(2)} \xrightarrow{a_3^{(2)}} \dots \\ \dots & & & & & & \end{array}$$

Here, $s_i^{(j)}$ is the state we were at time i of trial j , and $a_i^{(j)}$ is the corresponding action that was taken from that state. In practice, each of the trials above might be run until the MDP terminates (such as if the pole falls over in the inverted pendulum problem), or it might be run for some large but finite number of timesteps.

Given this “experience” in the MDP consisting of a number of trials, we can then easily derive the maximum likelihood estimates for the state transition probabilities:

$$P_{sa}(s') = \frac{\text{\#times took we action } a \text{ in state } s \text{ and got to } s'}{\text{\#times we took action } a \text{ in state } s} \quad (4)$$

Or, if the ratio above is “0/0”—corresponding to the case of never having taken action a in state s before—the we might simply estimate $P_{sa}(s')$ to be $1/|S|$. (I.e., estimate P_{sa} to be the uniform distribution over all states.)

Note that, if we gain more experience (observe more trials) in the MDP, there is an efficient way to update our estimated state transition probabilities

using the new experience. Specifically, if we keep around the counts for both the numerator and denominator terms of (4), then as we observe more trials, we can simply keep accumulating those counts. Computing the ratio of these counts then gives our estimate of P_{sa} .

Using a similar procedure, if R is unknown, we can also pick our estimate of the expected immediate reward $R(s)$ in state s to be the average reward observed in state s .

Having learned a model for the MDP, we can then use either value iteration or policy iteration to solve the MDP using the estimated transition probabilities and rewards. For example, putting together model learning and value iteration, here is one possible algorithm for learning in an MDP with unknown state transition probabilities:

1. Initialize π randomly.
2. Repeat {
 - (a) Execute π in the MDP for some number of trials.
 - (b) Using the accumulated experience in the MDP, update our estimates for P_{sa} (and R , if applicable).
 - (c) Apply value iteration with the estimated state transition probabilities and rewards to get a new estimated value function V .
 - (d) Update π to be the greedy policy with respect to V .

We note that, for this particular algorithm, there is one simple optimization that can make it run much more quickly. Specifically, in the inner loop of the algorithm where we apply value iteration, if instead of initializing value iteration with $V = 0$, we initialize it with the solution found during the previous iteration of our algorithm, then that will provide value iteration with a much better initial starting point and make it converge more quickly.

4 Continuous state MDPs

So far, we've focused our attention on MDPs with a finite number of states. We now discuss algorithms for MDPs that may have an infinite number of states. For example, for a car, we might represent the state as $(x, y, \theta, \dot{x}, \dot{y}, \dot{\theta})$, comprising its position (x, y) ; orientation θ ; velocity in the x and y directions $\dot{x}$ and $\dot{y}$; and angular velocity $\dot{\theta}$. Hence, $S = \mathbb{R}^6$ is an infinite set of states,

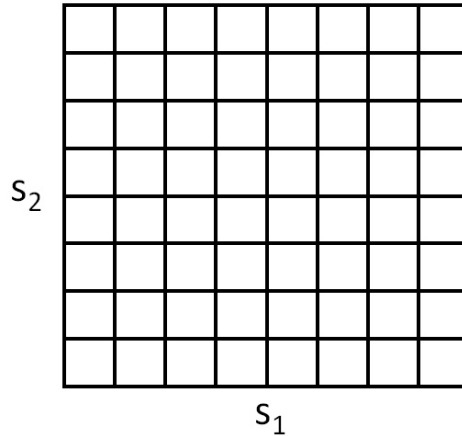
because there is an infinite number of possible positions and orientations for the car.² Similarly, the inverted pendulum you saw in PS4 has states $(x, \theta, \dot{x}, \dot{\theta})$, where θ is the angle of the pole. And, a helicopter flying in 3d space has states of the form $(x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi})$, where here the roll ϕ , pitch θ , and yaw ψ angles specify the 3d orientation of the helicopter.

In this section, we will consider settings where the state space is $S = \mathbb{R}^n$, and describe ways for solving such MDPs.

4.1 Discretization

Perhaps the simplest way to solve a continuous-state MDP is to discretize the state space, and then to use an algorithm like value iteration or policy iteration, as described previously.

For example, if we have 2d states (s_1, s_2) , we can use a grid to discretize the state space:



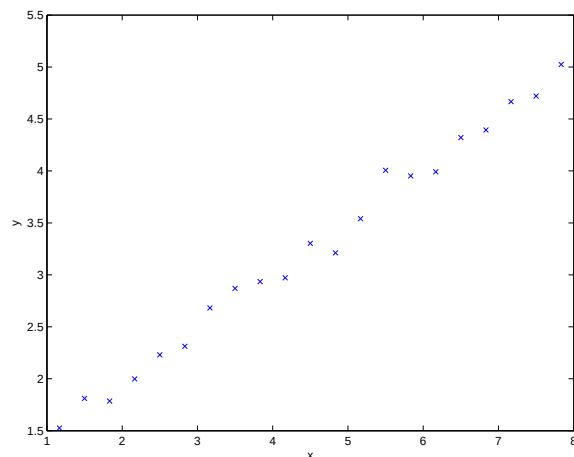
Here, each grid cell represents a separate discrete state $\bar{s}$. We can then approximate the continuous-state MDP via a discrete-state one $(\bar{S}, A, \{P_{\bar{s}a}\}, \gamma, R)$, where $\bar{S}$ is the set of discrete states, $\{P_{\bar{s}a}\}$ are our state transition probabilities over the discrete states, and so on. We can then use value iteration or policy iteration to solve for the $V^*(\bar{s})$ and $\pi^*(\bar{s})$ in the discrete state MDP $(\bar{S}, A, \{P_{\bar{s}a}\}, \gamma, R)$. When our actual system is in some continuous-valued state $s \in S$ and we need to pick an action to execute, we compute the corresponding discretized state $\bar{s}$, and execute action $\pi^*(\bar{s})$.

This discretization approach can work well for many problems. However, there are two downsides. First, it uses a fairly naive representation for V^*

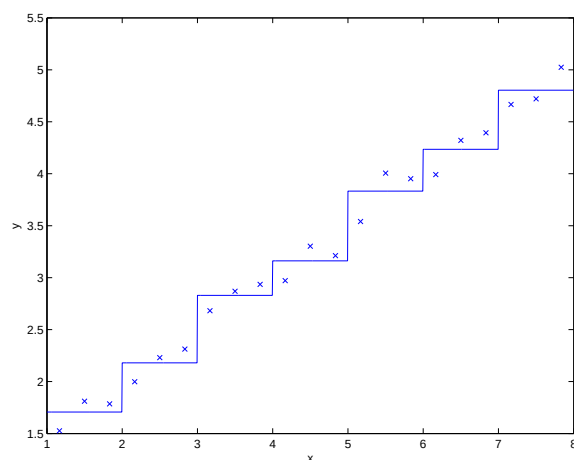
²Technically, θ is an orientation and so the range of θ is better written $\theta \in [-\pi, \pi)$ than $\theta \in \mathbb{R}$; but for our purposes, this distinction is not important.

(and π^*). Specifically, it assumes that the value function takes a constant value over each of the discretization intervals (i.e., that the value function is piecewise constant in each of the gridcells).

To better understand the limitations of such a representation, consider a *supervised learning* problem of fitting a function to this dataset:



Clearly, linear regression would do fine on this problem. However, if we instead discretize the x -axis, and then use a representation that is piecewise constant in each of the discretization intervals, then our fit to the data would look like this:



This piecewise constant representation just isn't a good representation for many smooth functions. It results in little smoothing over the inputs, and no generalization over the different grid cells. Using this sort of representation, we would also need a very fine discretization (very small grid cells) to get a good approximation.

A second downside of this representation is called the **curse of dimensionality**. Suppose $S = \mathbb{R}^n$, and we discretize each of the n dimensions of the state into k values. Then the total number of discrete states we have is k^n . This grows exponentially quickly in the dimension of the state space n , and thus does not scale well to large problems. For example, with a 10d state, if we discretize each state variable into 100 values, we would have $100^{10} = 10^{20}$ discrete states, which is far too many to represent even on a modern desktop computer.

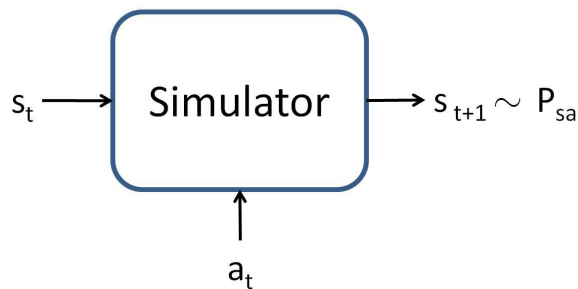
As a rule of thumb, discretization usually works extremely well for 1d and 2d problems (and has the advantage of being simple and quick to implement). Perhaps with a little bit of cleverness and some care in choosing the discretization method, it often works well for problems with up to 4d states. If you're extremely clever, and somewhat lucky, you may even get it to work for some 6d problems. But it very rarely works for problems any higher dimensional than that.

4.2 Value function approximation

We now describe an alternative method for finding policies in continuous-state MDPs, in which we approximate V^* directly, without resorting to discretization. This approach, called value function approximation, has been successfully applied to many RL problems.

4.2.1 Using a model or simulator

To develop a value function approximation algorithm, we will assume that we have a **model**, or **simulator**, for the MDP. Informally, a simulator is a black-box that takes as input any (continuous-valued) state s_t and action a_t , and outputs a next-state s_{t+1} sampled according to the state transition probabilities $P_{s_t a_t}$:



There're several ways that one can get such a model. One is to use physics simulation. For example, the simulator for the inverted pendulum

in PS4 was obtained by using the laws of physics to calculate what position and orientation the cart/pole will be in at time $t + 1$, given the current state at time t and the action a taken, assuming that we know all the parameters of the system such as the length of the pole, the mass of the pole, and so on. Alternatively, one can also use an off-the-shelf physics simulation software package which takes as input a complete physical description of a mechanical system, the current state s_t and action a_t , and computes the state s_{t+1} of the system a small fraction of a second into the future.³

An alternative way to get a model is to learn one from data collected in the MDP. For example, suppose we execute m **trials** in which we repeatedly take actions in an MDP, each trial for T timesteps. This can be done picking actions at random, executing some specific policy, or via some other way of choosing actions. We would then observe m state sequences like the following:

$$\begin{aligned} s_0^{(1)} &\xrightarrow{a_0^{(1)}} s_1^{(1)} \xrightarrow{a_1^{(1)}} s_2^{(1)} \xrightarrow{a_2^{(1)}} \dots \xrightarrow{a_{T-1}^{(1)}} s_T^{(1)} \\ s_0^{(2)} &\xrightarrow{a_0^{(2)}} s_1^{(2)} \xrightarrow{a_1^{(2)}} s_2^{(2)} \xrightarrow{a_2^{(2)}} \dots \xrightarrow{a_{T-1}^{(2)}} s_T^{(2)} \\ &\dots \\ s_0^{(m)} &\xrightarrow{a_0^{(m)}} s_1^{(m)} \xrightarrow{a_1^{(m)}} s_2^{(m)} \xrightarrow{a_2^{(m)}} \dots \xrightarrow{a_{T-1}^{(m)}} s_T^{(m)} \end{aligned}$$

We can then apply a learning algorithm to predict s_{t+1} as a function of s_t and a_t .

For example, one may choose to learn a linear model of the form

$$s_{t+1} = As_t + Ba_t, \tag{5}$$

using an algorithm similar to linear regression. Here, the parameters of the model are the matrices A and B , and we can estimate them using the data collected from our m trials, by picking

$$\arg \min_{A,B} \sum_{i=1}^m \sum_{t=0}^{T-1} \left\| s_{t+1}^{(i)} - \left(As_t^{(i)} + Ba_t^{(i)} \right) \right\|^2.$$

(This corresponds to the maximum likelihood estimate of the parameters.)

Having learned A and B , one option is to build a **deterministic** model, in which given an input s_t and a_t , the output s_{t+1} is exactly determined.

³Open Dynamics Engine (<http://www.ode.com>) is one example of a free/open-source physics simulator that can be used to simulate systems like the inverted pendulum, and that has been a reasonably popular choice among RL researchers.

Specifically, we always compute s_{t+1} according to Equation (5). Alternatively, we may also build a **stochastic** model, in which s_{t+1} is a random function of the inputs, by modelling it as

$$s_{t+1} = As_t + Ba_t + \epsilon_t,$$

where here ϵ_t is a noise term, usually modeled as $\epsilon_t \sim \mathcal{N}(0, \Sigma)$. (The covariance matrix Σ can also be estimated from data in a straightforward way.)

Here, we've written the next-state s_{t+1} as a linear function of the current state and action; but of course, non-linear functions are also possible. Specifically, one can learn a model $s_{t+1} = A\phi_s(s_t) + B\phi_a(a_t)$, where ϕ_s and ϕ_a are some non-linear feature mappings of the states and actions. Alternatively, one can also use non-linear learning algorithms, such as locally weighted linear regression, to learn to estimate s_{t+1} as a function of s_t and a_t . These approaches can also be used to build either deterministic or stochastic simulators of an MDP.

4.2.2 Fitted value iteration

We now describe the **fitted value iteration** algorithm for approximating the value function of a continuous state MDP. In the sequel, we will assume that the problem has a continuous state space $S = \mathbb{R}^n$, but that the action space A is small and discrete.⁴

Recall that in value iteration, we would like to perform the update

$$V(s) := R(s) + \gamma \max_a \int_{s'} P_{sa}(s') V(s') ds' \quad (6)$$

$$= R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}} [V(s')] \quad (7)$$

(In Section 2, we had written the value iteration update with a summation $V(s) := R(s) + \gamma \max_a \sum_{s'} P_{sa}(s') V(s')$ rather than an integral over states; the new notation reflects that we are now working in continuous states rather than discrete states.)

The main idea of fitted value iteration is that we are going to approximately carry out this step, over a finite sample of states $s^{(1)}, \dots, s^{(m)}$. Specifically, we will use a supervised learning algorithm—linear regression in our

⁴In practice, most MDPs have much smaller action spaces than state spaces. E.g., a car has a 6d state space, and a 2d action space (steering and velocity controls); the inverted pendulum has a 4d state space, and a 1d action space; a helicopter has a 12d state space, and a 4d action space. So, discretizing this set of actions is usually less of a problem than discretizing the state space would have been.

description below—to approximate the value function as a linear or non-linear function of the states:

$$V(s) = \theta^T \phi(s).$$

Here, ϕ is some appropriate feature mapping of the states.

For each state s in our finite sample of m states, fitted value iteration will first compute a quantity $y^{(i)}$, which will be our approximation to $R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}}[V(s')]$ (the right hand side of Equation 7). Then, it will apply a supervised learning algorithm to try to get $V(s)$ close to $R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}}[V(s')]$ (or, in other words, to try to get $V(s)$ close to $y^{(i)}$).

In detail, the algorithm is as follows:

1. Randomly sample m states $s^{(1)}, s^{(2)}, \dots, s^{(m)} \in S$.
2. Initialize $\theta := 0$.
3. Repeat {

For $i = 1, \dots, m$ {

For each action $a \in A$ {

Sample $s'_1, \dots, s'_k \sim P_{s^{(i)}a}$ (using a model of the MDP).

Set $q(a) = \frac{1}{k} \sum_{j=1}^k R(s^{(i)}) + \gamma V(s'_j)$

// Hence, $q(a)$ is an estimate of $R(s^{(i)}) + \gamma \mathbb{E}_{s' \sim P_{s^{(i)}a}}[V(s')]$.

Set $y^{(i)} = \max_a q(a)$.

// Hence, $y^{(i)}$ is an estimate of $R(s^{(i)}) + \gamma \max_a \mathbb{E}_{s' \sim P_{s^{(i)}a}}[V(s')]$.

// In the original value iteration algorithm (over discrete states)

// we updated the value function according to $V(s^{(i)}) := y^{(i)}$.

// In this algorithm, we want $V(s^{(i)}) \approx y^{(i)}$, which we'll achieve

// using supervised learning (linear regression).

Set $\theta := \arg \min_{\theta} \frac{1}{2} \sum_{i=1}^m (\theta^T \phi(s^{(i)}) - y^{(i)})^2$

}

Above, we had written out fitted value iteration using linear regression as the algorithm to try to make $V(s^{(i)})$ close to $y^{(i)}$. That step of the algorithm is completely analogous to a standard supervised learning (regression) problem in which we have a training set $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$, and want to learn a function mapping from x to y ; the only difference is that here s plays the role of x . Even though our description above used linear regression, clearly other regression algorithms (such as locally weighted linear regression) can also be used.

Unlike value iteration over a discrete set of states, fitted value iteration cannot be proved to always converge. However, in practice, it often does converge (or approximately converge), and works well for many problems. Note also that if we are using a deterministic simulator/model of the MDP, then fitted value iteration can be simplified by setting $k = 1$ in the algorithm. This is because the expectation in Equation (7) becomes an expectation over a deterministic distribution, and so a single example is sufficient to exactly compute that expectation. Otherwise, in the algorithm above, we had to draw k samples, and average to try to approximate that expectation (see the definition of $q(a)$, in the algorithm pseudo-code).

Finally, fitted value iteration outputs V , which is an approximation to V^* . This implicitly defines our policy. Specifically, when our system is in some state s , and we need to choose an action, we would like to choose the action

$$\arg \max_a \mathbb{E}_{s' \sim P_{sa}} [V(s')] \quad (8)$$

The process for computing/approximating this is similar to the inner-loop of fitted value iteration, where for each action, we sample $s'_1, \dots, s'_k \sim P_{sa}$ to approximate the expectation. (And again, if the simulator is deterministic, we can set $k = 1$.)

In practice, there're often other ways to approximate this step as well. For example, one very common case is if the simulator is of the form $s_{t+1} = f(s_t, a_t) + \epsilon_t$, where f is some deterministic function of the states (such as $f(s_t, a_t) = As_t + Ba_t$), and ϵ is zero-mean Gaussian noise. In this case, we can pick the action given by

$$\arg \max_a V(f(s, a)).$$

In other words, here we are just setting $\epsilon_t = 0$ (i.e., ignoring the noise in the simulator), and setting $k = 1$. Equivalently, this can be derived from Equation (8) using the approximation

$$\mathbb{E}_{s'} [V(s')] \approx V(\mathbb{E}_{s'} [s']) \quad (9)$$

$$= V(f(s, a)), \quad (10)$$

where here the expectation is over the random $s' \sim P_{sa}$. So long as the noise terms ϵ_t are small, this will usually be a reasonable approximation.

However, for problems that don't lend themselves to such approximations, having to sample $k|A|$ states using the model, in order to approximate the expectation above, can be computationally expensive.

CS229 Lecture notes

Dan Boneh & Andrew Ng

Part XIV

LQR, DDP and LQG

Linear Quadratic Regulation, Differential Dynamic Programming and Linear Quadratic Gaussian

1 Finite-horizon MDPs

In the previous set of notes about Reinforcement Learning, we defined Markov Decision Processes (MDPs) and covered Value Iteration / Policy Iteration in a simplified setting. More specifically we introduced the **optimal Bellman equation** that defines the optimal value function V^{π^*} of the optimal policy π^* .

$$V^{\pi^*}(s) = R(s) + \max_{a \in \mathcal{A}} \gamma \sum_{s' \in \mathcal{S}} P_{sa}(s') V^{\pi^*}(s')$$

Recall that from the optimal value function, we were able to recover the optimal policy π^* with

$$\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P_{sa}(s') V^*(s')$$

In this set of lecture notes we'll place ourselves in a more general setting:

1. We want to write equations that make sense for both the discrete and the continuous case. We'll therefore write

scribe: Guillaume Genthial

$$\mathbb{E}_{s' \sim P_{sa}} [V^{\pi^*}(s')] \quad \text{instead of} \\ \sum_{s' \in S} P_{sa}(s') V^{\pi^*}(s')$$

meaning that we take the expectation of the value function at the next state. In the finite case, we can rewrite the expectation as a sum over states. In the continuous case, we can rewrite the expectation as an integral. The notation $s' \sim P_{sa}$ means that the state s' is sampled from the distribution P_{sa} .

2. We'll assume that the rewards depend on **both states and actions**. In other words, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. This implies that the previous mechanism for computing the optimal action is changed into

$$\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} R(s, a) + \gamma \mathbb{E}_{s' \sim P_{sa}} [V^{\pi^*}(s')]$$

3. Instead of considering an infinite horizon MDP, we'll assume that we have a **finite horizon MDP** that will be defined as a tuple

$$(\mathcal{S}, \mathcal{A}, P_{sa}, T, R)$$

with $T > 0$ the **time horizon** (for instance $T = 100$). In this setting, our definition of payoff is going to be (slightly) different:

$$R(s_0, a_0) + R(s_1, a_1) + \dots + R(s_T, a_T)$$

instead of (infinite horizon case)

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \dots \\ \sum_{t=0}^{\infty} R(s_t, a_t) \gamma^t$$

What happened to the discount factor γ ? Remember that the introduction of γ was (partly) justified by the necessity of making sure that

the infinite sum would be finite and well-defined. If the rewards are bounded by a constant $\bar{R}$, the payoff is indeed bounded by

$$|\sum_{t=0}^{\infty} R(s_t)\gamma^t| \leq \bar{R} \sum_{t=0}^{\infty} \gamma^t$$

and we recognize a geometric sum! Here, as the payoff is a finite sum, the discount factor γ is not necessary anymore.

In this new setting, things behave quite differently. First, the optimal policy π^* might be non-stationary, meaning that **it changes over time**. In other words, now we have

$$\pi^{(t)} : \mathcal{S} \rightarrow \mathcal{A}$$

where the superscript (t) denotes the policy at time step t . The dynamics of the finite horizon MDP following policy $\pi^{(t)}$ proceeds as follows: we start in some state s_0 , take some action $a_0 := \pi^{(0)}(s_0)$ according to our policy at time step 0. The MDP transitions to a successor s_1 , drawn according to $P_{s_0 a_0}$. Then, we get to pick another action $a_1 := \pi^{(1)}(s_1)$ following our new policy at time step 1 and so on...

Why does the optimal policy happen to be non-stationary in the finite-horizon setting? Intuitively, as we have a finite numbers of actions to take, we might want to adopt different strategies depending on where we are in the environment and how much time we have left. Imagine a grid with 2 goals with rewards +1 and +10. At the beginning, we might want to take actions to aim for the +10 goal. But if after some steps, dynamics somehow pushed us closer to the +1 goal and we don't have enough steps left to be able to reach the +10 goal, then a better strategy would be to aim for the +1 goal...

4. This observation allows us to use **time dependent dynamics**

$$s_{t+1} \sim P_{s_t, a_t}^{(t)}$$

meaning that the transition's distribution $P_{s_t, a_t}^{(t)}$ changes over time. The same thing can be said about $R^{(t)}$. Note that this setting is a better

model for real life. In a car, the gas tank empties, traffic changes, etc. Combining the previous remarks, we'll use the following general formulation for our finite horizon MDP

$$(\mathcal{S}, \mathcal{A}, P_{sa}^{(t)}, T, R^{(t)})$$

Remark: notice that the above formulation would be equivalent to adding the time into the state.

The value function at time t for a policy π is then defined in the same way as before, as an expectation over trajectories generated following policy π starting in state s .

$$V_t(s) = \mathbb{E} [R^{(t)}(s_t, a_t) + \dots + R^{(T)}(s_T, a_T) | s_t = s, \pi]$$

Now, the question is

In this finite-horizon setting, how do we find the optimal value function

$$V_t^*(s) = \max_{\pi} V_t^{\pi}(s)$$

It turns out that Bellman's equation for Value Iteration is made for **Dynamic Programming**. This may come as no surprise as Bellman is one of the fathers of dynamic programming and the Bellman equation is strongly related to the field. To understand how we can simplify the problem by adopting an iteration-based approach, we make the following observations:

1. Notice that at the end of the game (for time step T), the optimal value is obvious

$$\forall s \in \mathcal{S} : V_T^*(s) := \max_{a \in \mathcal{A}} R^{(T)}(s, a) \quad (1)$$

2. For another time step $0 \leq t < T$, if we suppose that we know the optimal value function for the next time step V_{t+1}^* , then we have

$$\forall t < T, s \in \mathcal{S} : V_t^*(s) := \max_{a \in \mathcal{A}} \left[R^{(t)}(s, a) + \mathbb{E}_{s' \sim P_{sa}^{(t)}} [V_{t+1}^*(s')] \right] \quad (2)$$

With these observations in mind, we can come up with a clever algorithm to solve for the optimal value function:

1. compute V_T^* using equation (1).

2. for $t = T - 1, \dots, 0$:

compute V_t^* using V_{t+1}^* using equation (2)

Side note We can interpret standard value iteration as a special case of this general case, but without keeping track of time. It turns out that in the standard setting, if we run value iteration for T steps, we get a γ^T approximation of the optimal value iteration (geometric convergence). See problem set 4 for a proof of the following result:

Theorem Let B denote the Bellman update and $\|f(x)\|_\infty := \sup_x |f(x)|$. If V_t denotes the value function at the t -th step, then

$$\begin{aligned} \|V_{t+1} - V^*\|_\infty &= \|B(V_t) - V^*\|_\infty \\ &\leq \gamma \|V_t - V^*\|_\infty \\ &\leq \gamma^t \|V_1 - V^*\|_\infty \end{aligned}$$

In other words, the Bellman operator B is a γ -contracting operator.

2 Linear Quadratic Regulation (LQR)

In this section, we'll cover a special case of the finite-horizon setting described in Section 1, for which the **exact solution** is (easily) tractable. This model is widely used in robotics, and a common technique in many problems is to reduce the formulation to this framework.

First, let's describe the model's assumptions. We place ourselves in the continuous setting, with

$$\mathcal{S} = \mathbb{R}^n, \quad \mathcal{A} = \mathbb{R}^d$$

and we'll assume **linear transitions** (with noise)

$$s_{t+1} = A_t s_t + B_t a_t + w_t$$

where $A_t \in \mathbb{R}^{n \times n}$, $B_t \in \mathbb{R}^{n \times d}$ are matrices and $w_t \sim \mathcal{N}(0, \Sigma_t)$ is some gaussian noise (with **zero** mean). As we'll show in the following paragraphs,

it turns out that the noise, as long as it has zero mean, does not impact the optimal policy!

We'll also assume **quadratic rewards**

$$R^{(t)}(s_t, a_t) = -s_t^\top U_t s_t - a_t^\top W_t a_t$$

where $U_t \in R^{n \times n}$, $W_t \in R^{d \times d}$ are positive definite matrices (meaning that the reward is always **negative**).

Remark Note that the quadratic formulation of the reward is equivalent to saying that we want our state to be close to the origin (where the reward is higher). For example, if $U_t = I_n$ (the identity matrix) and $W_t = I_d$, then $R_t = -\|s_t\|^2 - \|a_t\|^2$, meaning that we want to take smooth actions (small norm of a_t) to go back to the origin (small norm of s_t). This could model a car trying to stay in the middle of lane without making impulsive moves...

Now that we have defined the assumptions of our LQR model, let's cover the 2 steps of the LQR algorithm

- step 1** suppose that we don't know the matrices A, B, Σ . To estimate them, we can follow the ideas outlined in the Value Approximation section of the RL notes. First, collect transitions from an arbitrary policy. Then, use linear regression to find $\operatorname{argmin}_{A,B} \sum_{i=1}^m \sum_{t=0}^{T-1} \left\| s_{t+1}^{(i)} - \left(A s_t^{(i)} + B a_t^{(i)} \right) \right\|^2$. Finally, use a technique seen in Gaussian Discriminant Analysis to learn Σ .
- step 2** assuming that the parameters of our model are known (given or estimated with step 1), we can derive the optimal policy using dynamic programming.

In other words, given

$$\begin{cases} s_{t+1} &= A_t s_t + B_t a_t + w_t & A_t, B_t, U_t, W_t, \Sigma_t \text{ known} \\ R^{(t)}(s_t, a_t) &= -s_t^\top U_t s_t - a_t^\top W_t a_t \end{cases}$$

we want to compute V_t^* . If we go back to section 1, we can apply dynamic programming, which yields

1. Initialization step

For the last time step T ,

$$\begin{aligned}
V_T^*(s_T) &= \max_{a_T \in \mathcal{A}} R_T(s_T, a_T) \\
&= \max_{a_T \in \mathcal{A}} -s_T^\top U_T s_T - a_T^\top W_t a_T \\
&= -s_T^\top U_t s_T \quad (\text{maximized for } a_T = 0)
\end{aligned}$$

2. Recurrence step

Let $t < T$. Suppose we know V_{t+1}^* .

Fact 1: It can be shown that if V_{t+1}^* is a quadratic function in s_t , then V_t^* is also a quadratic function. In other words, there exists some matrix Φ and some scalar Ψ such that

$$\begin{aligned}
&\text{if } V_{t+1}^*(s_{t+1}) = s_{t+1}^\top \Phi_{t+1} s_{t+1} + \Psi_{t+1} \\
&\text{then } V_t^*(s_t) = s_t^\top \Phi_t s_t + \Psi_t
\end{aligned}$$

For time step $t = T$, we had $\Phi_t = -U_T$ and $\Psi_T = 0$.

Fact 2: We can show that the optimal policy is just a linear function of the state.

Knowing V_{t+1}^* is equivalent to knowing Φ_{t+1} and Ψ_{t+1} , so we just need to explain how we compute Φ_t and Ψ_t from Φ_{t+1} and Ψ_{t+1} and the other parameters of the problem.

$$\begin{aligned}
V_t^*(s_t) &= s_t^\top \Phi_t s_t + \Psi_t \\
&= \max_{a_t} \left[R^{(t)}(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim P_{s_t, a_t}^{(t)}} [V_{t+1}^*(s_{t+1})] \right] \\
&= \max_{a_t} \left[-s_t^\top U_t s_t - a_t^\top V_t a_t + \mathbb{E}_{s_{t+1} \sim \mathcal{N}(A_t s_t + B_t a_t, \Sigma_t)} [s_{t+1}^\top \Phi_{t+1} s_{t+1} + \Psi_{t+1}] \right]
\end{aligned}$$

where the second line is just the definition of the optimal value function and the third line is obtained by plugging in the dynamics of our model along with the quadratic assumption. Notice that the last expression is a quadratic function in a_t and can thus be (easily) optimized¹. We get the optimal action a_t^*

¹Use the identity $\mathbb{E} [w_t^\top \Phi_{t+1} w_t] = \text{Tr}(\Sigma_t \Phi_{t+1})$ with $w_t \sim \mathcal{N}(0, \Sigma_t)$

$$\begin{aligned} a_t^* &= [(B_t^\top \Phi_{t+1} B_t - V_t)^{-1} B_t \Phi_{t+1} A_t] \cdot s_t \\ &= L_t \cdot s_t \end{aligned}$$

where

$$L_t := [(B_t^\top \Phi_{t+1} B_t - W_t)^{-1} B_t \Phi_{t+1} A_t]$$

which is an impressive result: our optimal policy is **linear in** s_t . Given a_t^* we can solve for Φ_t and Ψ_t . We finally get the **Discrete Ricatti equations**

$$\begin{aligned} \Phi_t &= A_t^\top \left(\Phi_{t+1} - \Phi_{t+1} B_t (B_t^\top \Phi_{t+1} B_t - W_t)^{-1} B_t \Phi_{t+1} \right) A_t - U_t \\ \Psi_t &= -\text{tr}(\Sigma_t \Phi_{t+1}) + \Psi_{t+1} \end{aligned}$$

Fact 3: we notice that Φ_t depends on neither Ψ nor the noise Σ_t ! As L_t is a function of A_t, B_t and Φ_{t+1} , it implies that the optimal policy also **does not depend on the noise!** (But Ψ_t does depend on Σ_t , which implies that V_t^* depends on Σ_t .)

Then, to summarize, the LQR algorithm works as follows

1. (if necessary) estimate parameters A_t, B_t, Σ_t
2. initialize $\Phi_T := -U_T$ and $\Psi_T := 0$.
3. iterate from $t = T - 1 \dots 0$ to update Φ_t and Ψ_t using Φ_{t+1} and Ψ_{t+1} using the discrete Ricatti equations. If there exists a policy that drives the state towards zero, then convergence is guaranteed!

Using Fact 3, we can be even more clever and make our algorithm run (slightly) faster! As the optimal policy does not depend on Ψ_t , and the update of Φ_t only depends on Φ_t , it is sufficient to update **only** Φ_t !

3 From non-linear dynamics to LQR

It turns out that a lot of problems can be reduced to LQR, even if dynamics are non-linear. While LQR is a nice formulation because we are able to come up with a nice exact solution, it is far from being general. Let's take for instance the case of the inverted pendulum. The transitions between states look like

$$\begin{pmatrix} x_{t+1} \\ \dot{x}_{t+1} \\ \theta_{t+1} \\ \dot{\theta}_{t+1} \end{pmatrix} = F \left(\begin{pmatrix} x_t \\ \dot{x}_t \\ \theta_t \\ \dot{\theta}_t \end{pmatrix}, a_t \right)$$

where the function F depends on the cos of the angle etc. Now, the question we may ask is

Can we linearize this system?

3.1 Linearization of dynamics

Let's suppose that at time t , the system spends most of its time in some state $\bar{s}_t$ and the actions we perform are around $\bar{a}_t$. For the inverted pendulum, if we reached some kind of optimal, this is true: our actions are small and we don't deviate much from the vertical.

We are going to use Taylor expansion to linearize the dynamics. In the simple case where the state is one-dimensional and the transition function F does not depend on the action, we would write something like

$$s_{t+1} = F(s_t) \approx F(\bar{s}_t) + F'(\bar{s}_t) \cdot (s_t - \bar{s}_t)$$

In the more general setting, the formula looks the same, with gradients instead of simple derivatives

$$s_{t+1} \approx F(\bar{s}_t, \bar{a}_t) + \nabla_s F(\bar{s}_t, \bar{a}_t) \cdot (s_t - \bar{s}_t) + \nabla_a F(\bar{s}_t, \bar{a}_t) \cdot (a_t - \bar{a}_t) \quad (3)$$

and now, s_{t+1} is linear in s_t and a_t , because we can rewrite equation (3) as

$$s_{t+1} \approx As_t + Bs_t + \kappa$$

where κ is some constant and A, B are matrices. Now, this writing looks awfully similar to the assumptions made for LQR. We just have to get rid

of the constant term κ ! It turns out that the constant term can be absorbed into s_t by artificially increasing the dimension by one. This is the same trick that we used at the beginning of the class for linear regression...

3.2 Differential Dynamic Programming (DDP)

The previous method works well for cases where the goal is to stay around some state s^* (think about the inverted pendulum, or a car having to stay in the middle of a lane). However, in some cases, the goal can be more complicated.

We'll cover a method that applies when our system has to follow some trajectory (think about a rocket). This method is going to discretize the trajectory into discrete time steps, and create intermediary goals around which we will be able to use the previous technique! This method is called **Differential Dynamic Programming**. The main steps are

step 1 come up with a nominal trajectory using a naive controller, that approximate the trajectory we want to follow. In other words, our controller is able to approximate the gold trajectory with

$$s_0^*, a_0^* \rightarrow s_1^*, a_1^* \rightarrow \dots$$

step 2 linearize the dynamics around each trajectory point s_t^* , in other words

$$s_{t+1} \approx F(s_t^*, a_t^*) + \nabla_s F(s_t^*, a_t^*)(s_t - s_t^*) + \nabla_a F(s_t^*, a_t^*)(a_t - a_t^*)$$

where s_t, a_t would be our current state and action. Now that we have a linear approximation around each of these points, we can use the previous section and rewrite

$$s_{t+1} = A_t \cdot s_t + B_t \cdot a_t$$

(notice that in that case, we use the non-stationary dynamics setting that we mentioned at the beginning of these lecture notes)

Note We can apply a similar derivation for the reward $R^{(t)}$, with a second-order Taylor expansion.

$$\begin{aligned}
R(s_t, a_t) &\approx R(s_t^*, a_t^*) + \nabla_s R(s_t^*, a_t^*)(s_t - s_t^*) + \nabla_a R(s_t^*, a_t^*)(a_t - a_t^*) \\
&+ \frac{1}{2}(s_t - s_t^*)^\top H_{ss}(s_t - s_t^*) + (s_t - s_t^*)^\top H_{sa}(a_t - a_t^*) \\
&+ \frac{1}{2}(a_t - a_t^*)^\top H_{aa}(a_t - a_t^*)
\end{aligned}$$

where H_{xy} refers to the entry of the Hessian of R with respect to x and y evaluated in (s_t^*, a_t^*) (omitted for readability). This expression can be re-written as

$$R_t(s_t, a_t) = -s_t^\top U_t s_t - a_t^\top W_t a_t$$

for some matrices U_t, W_t , with the same trick of adding an extra dimension of ones. To convince yourself, notice that

$$\begin{pmatrix} 1 & x \end{pmatrix} \cdot \begin{pmatrix} a & b \\ b & c \end{pmatrix} \cdot \begin{pmatrix} 1 \\ x \end{pmatrix} = a + 2bx + cx^2$$

step 3 Now, you can convince yourself that our problem is **strictly** re-written in the LQR framework. Let's just use LQR to find the optimal policy π_t . As a result, our new controller will (hopefully) be better!

Note: Some problems might arise if the LQR trajectory deviates too much from the linearized approximation of the trajectory, but that can be fixed with reward-shaping...

step 4 Now that we get a new controller (our new policy π_t), we use it to produce a new trajectory

$$s_0^*, \pi_0(s_0^*) \rightarrow s_1^*, \pi_1(s_1^*) \rightarrow \dots \rightarrow s_T^*$$

note that when we generate this new trajectory, we use the real F and not its linear approximation to compute transitions, meaning that

$$s_{t+1}^* = F(s_t^*, a_t^*)$$

then, go back to step 2 and repeat until some stopping criterion.

4 Linear Quadratic Gaussian (LQG)

Often, in the real world, we don't get to observe the full state s_t . For example, an autonomous car could receive an image from a camera, which is merely an **observation**, and not the full state of the world. So far, we assumed that the state was available. As this might not hold true for most of the real-world problems, we need a new tool to model this situation: **Partially Observable MDPs**.

A POMDP is an MDP with an extra observation layer. In other words, we introduce a new variable o_t , that follows some conditional distribution given the current state s_t

$$o_t|s_t \sim O(o|s)$$

Formally, a finite-horizon POMDP is given by a tuple

$$(\mathcal{S}, \mathcal{O}, \mathcal{A}, P_{sa}, T, R)$$

Within this framework, the general strategy is to maintain a **belief state** (distribution over states) based on the observation $o_1, \dots, o_t$. Then, a policy in a POMDP maps belief states to actions.

In this section, we'll present an extension of LQR to this new setting. Assume that we observe $y_t \in \mathbb{R}^m$ with $m < n$ such that

$$\begin{cases} y_t &= C \cdot s_t + v_t \\ s_{t+1} &= A \cdot s_t + B \cdot a_t + w_t \end{cases}$$

where $C \in \mathbb{R}^{m \times n}$ is a compression matrix and v_t is the sensor noise (also gaussian, like w_t). Note that the reward function $R^{(t)}$ is left unchanged, as a function of the state (not the observation) and action. Also, as distributions are gaussian, the belief state is also going to be gaussian. In this new framework, let's give an overview of the strategy we are going to adopt to find the optimal policy:

step 1 first, compute the distribution on the possible states (the belief state), based on the observations we have. In other words, we want to compute the mean $s_{t|t}$ and the covariance $\Sigma_{t|t}$ of

$$s_t|y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t})$$

to perform the computation efficiently over time, we'll use the **Kalman Filter** algorithm (used on-board Apollo Lunar Module!).

step 2 now that we have the distribution, we'll use the mean $s_{t|t}$ as the best approximation for s_t

step 3 then set the action $a_t := L_t s_{t|t}$ where L_t comes from the regular LQR algorithm.

Intuitively, to understand why this works, notice that $s_{t|t}$ is a noisy approximation of s_t (equivalent to adding more noise to LQR) but we proved that LQR is independent of the noise!

Step 1 needs to be explicated. We'll cover a simple case where there is no action dependence in our dynamics (but the general case follows the same idea). Suppose that

$$\begin{cases} s_{t+1} &= A \cdot s_t + w_t, & w_t \sim N(0, \Sigma_s) \\ y_t &= C \cdot s_t + v_t, & v_t \sim N(0, \Sigma_y) \end{cases}$$

As noises are Gaussians, we can easily prove that the joint distribution is also Gaussian

$$\begin{pmatrix} s_1 \\ \vdots \\ s_t \\ y_1 \\ \vdots \\ y_t \end{pmatrix} \sim \mathcal{N}(\mu, \Sigma) \quad \text{for some } \mu, \Sigma$$

then, using the marginal formulas of gaussians (see Factor Analysis notes), we would get

$$s_t | y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t})$$

However, computing the marginal distribution parameters using these formulas would be computationally expensive! It would require manipulating matrices of shape $t \times t$. Recall that inverting a matrix can be done in $O(t^3)$, and it would then have to be repeated over the time steps, yielding a cost in $O(t^4)$!

The **Kalman filter** algorithm provides a much better way of computing the mean and variance, by updating them over time in **constant time in**

t ! The kalman filter is based on two basics steps. Assume that we know the distribution of $s_t|y_1, \dots, y_t$:

predict step compute $s_{t+1}|y_1, \dots, y_t$

update step compute $s_{t+1}|y_1, \dots, y_{t+1}$

and iterate over time steps! The combination of the predict and update steps updates our belief states. In other words, the process looks like

$$(s_t|y_1, \dots, y_t) \xrightarrow{\text{predict}} (s_{t+1}|y_1, \dots, y_t) \xrightarrow{\text{update}} (s_{t+1}|y_1, \dots, y_{t+1}) \xrightarrow{\text{predict}} \dots$$

predict step Suppose that we know the distribution of

$$s_t|y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t})$$

then, the distribution over the next state is also a gaussian distribution

$$s_{t+1}|y_1, \dots, y_t \sim \mathcal{N}(s_{t+1|t}, \Sigma_{t+1|t})$$

where

$$\begin{cases} s_{t+1|t} &= A \cdot s_{t|t} \\ \Sigma_{t+1|t} &= A \cdot \Sigma_{t|t} \cdot A^\top + \Sigma_s \end{cases}$$

update step given $s_{t+1|t}$ and $\Sigma_{t+1|t}$ such that

$$s_{t+1}|y_1, \dots, y_t \sim \mathcal{N}(s_{t+1|t}, \Sigma_{t+1|t})$$

we can prove that

$$s_{t+1}|y_1, \dots, y_{t+1} \sim \mathcal{N}(s_{t+1|t+1}, \Sigma_{t+1|t+1})$$

where

$$\begin{cases} s_{t+1|t+1} &= s_{t+1|t} + K_t(y_{t+1} - C s_{t+1|t}) \\ \Sigma_{t+1|t+1} &= \Sigma_{t+1|t} - K_t \cdot C \cdot \Sigma_{t+1|t} \end{cases}$$

with

$$K_t := \Sigma_{t+1|t} C^\top (C \Sigma_{t+1|t} C^\top + \Sigma_y)^{-1}$$

The matrix K_t is called the **Kalman gain**.

Now, if we have a closer look at the formulas, we notice that we don't need the observations prior to time step t ! The update steps only depends on the previous distribution. Putting it all together, the algorithm first runs a forward pass to compute the K_t , $\Sigma_{t|t}$ and $s_{t|t}$ (sometimes referred to as $\hat{s}$ in the literature). Then, it runs a backward pass (the LQR updates) to compute the quantities Ψ_t , $\bar{\Psi}_t$ and L_t . Finally, we recover the optimal policy with $a_t^* = L_t s_{t|t}$.

CS229 Bias-Variance and Error Analysis

Yoann Le Colonnec

October 2, 2017

1 The Bias-Variance Tradeoff

Assume you are given a well fitted machine learning model $\hat{f}$ that you want to apply on some test dataset. For instance, the model could be a linear regression whose parameters were computed using some training set different from your test set. For each point x in your test set, you want to predict the associated target $y \in \mathbb{R}$, and compute the mean squared error (MSE)

$$\mathbb{E}_{(x,y) \sim \text{test set}} |\hat{f}(x) - y|^2$$

You now realize that this MSE is too high, and try to find an explanation to this result:

- Overfitting: the model is too closely related to the examples in the training set and doesn't generalize well to other examples.
- Underfitting: the model didn't gather enough information from the training set, and doesn't capture the link between the features x and the target y .
- The data is simply noisy, that is the model is neither overfitting or underfitting, and the high MSE is simply due to the amount of noise in the dataset.

Our intuition can be formalized by the **Bias-Variance tradeoff**.

Assume that the points in your training/test set are all taken from a similar distribution, with

$$y_i = f(x_i) + \epsilon_i, \quad \text{where the noise } \epsilon_i \text{ satisfies } \mathbb{E}(\epsilon_i) = 0, \text{ Var}(\epsilon_i) = \sigma^2$$

and your goal is to compute f . By looking at your training set, you obtain an estimate $\hat{f}$. Now use this estimate with your test set, meaning that for each example j in the test set, your prediction for $y_j = f(x_j) + \epsilon_j$ is $\hat{f}(x_j)$. Here, x_j is a fixed real number (or vector if the feature space is multi-dimensional) thus $f(x_j)$ is fixed, and ϵ_j is a real random variable with mean 0 and variance σ^2 . The crucial observation is that $\hat{f}(x_j)$ is random since it depends on the values ϵ_i from the training set. That's why talking about the bias $\mathbb{E}(\hat{f}(x) - f(x))$ and the variance of $\hat{f}$ makes sense.

We can now compute our MSE on the test set by computing the following expectation with respect to the possible training sets (since $\hat{f}$ is a random variable function of the choice of the training set)

$$\begin{aligned}
\text{Test MSE} &= \mathbb{E} \left((y - \hat{f}(x))^2 \right) \\
&= \mathbb{E} \left((\epsilon + f(x) - \hat{f}(x))^2 \right) \\
&= \mathbb{E}(\epsilon^2) + \mathbb{E} \left((f(x) - \hat{f}(x))^2 \right) \\
&= \sigma^2 + \left(\mathbb{E}(f(x) - \hat{f}(x)) \right)^2 + \text{Var} \left(f(x) - \hat{f}(x) \right) \\
&= \sigma^2 + \left(\text{Bias } \hat{f}(x) \right)^2 + \text{Var} \left(\hat{f}(x) \right)
\end{aligned}$$

There is nothing we can do about the first term σ^2 as we can not predict the noise ϵ by definition. The bias term is due to underfitting, meaning that on average, $\hat{f}$ does not predict f . The last term is closely related to overfitting, the prediction $\hat{f}$ is too close from the values y_{train} and varies a lot with the choice of our training set.

To sum up, we can understand our MSE as follows

$$\begin{array}{lll}
\text{High Bias} & \longleftrightarrow & \text{Underfitting} \\
\text{High Variance} & \longleftrightarrow & \text{Overfitting} \\
\text{Large } \sigma^2 & \longleftrightarrow & \text{Noisy data}
\end{array}$$

Hence, when analyzing the performance of a machine learning algorithm, we must always ask ourselves how to reduce the bias without increasing the variance, and respectively how to reduce the variance without increasing the bias. Most of the time, reducing one will increase the other, and there is a tradeoff between bias and variance.

2 Error Analysis

Even though understanding whether our poor test error is due to high bias or high variance is important, knowing which parts of the machine learning algorithm lead to this error or score is crucial.

Consider the machine learning pipeline on figure 1.

The algorithms is divided into several steps

1. The inputs are taken from a camera image
2. Preprocessing to remove the background on the image. For instance, if the image are taken from a security camera, the background is always the same, and we could remove it easily by keeping the pixels that changed on the image.
3. Detect the position of the face.
4. Detect the eyes - Detect the nose - Detect the mouth

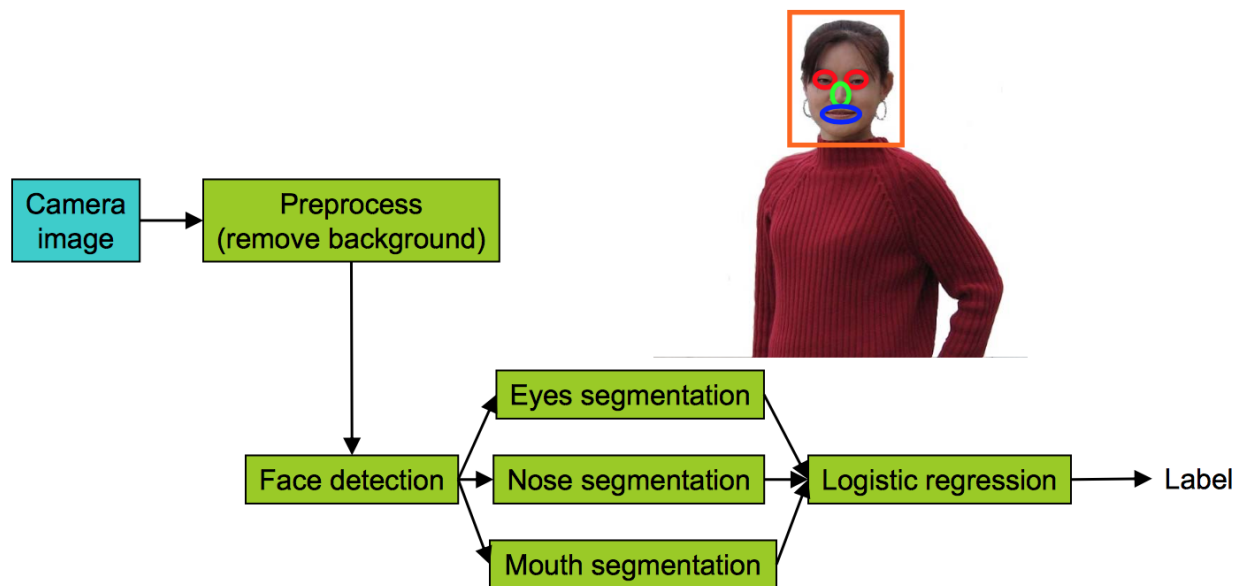


Figure 1: Face recognition pipeline

5. Final logistic regression step to predict the label

If you build a complicated system like this one, you might want to figure out how much error is attributable to each of the components, how good is each of these green boxes. Indeed, if one of these boxes is really problematic, you might want to spend more time trying to improve the performance of that one green box. How do you decide what part to focus on?

One thing we can do is plug in the ground-truth for each component, and see how accuracy changes. Let's say the overall accuracy of the system is 85% (pretty bad). You can now take your development set and manually give it the perfect background removal, that is, instead of using your background removal algorithm, manually specify the perfect background removal yourself (using photoshop for instance), and look at how much that affect the performance of the overall system.

Now let's say the accuracy only improves by 0.1%. This gives us an upperbound, that is even if we worked for years on background removal, it wouldn't help our system by more than 0.1%.

Now let's give the pipeline the perfect face detection by specifying the position of the face manually, see how much we improve the performance, and so on.

The results are specified in the table 1.

Looking at the table, we know that working on the background removal won't help much. It also tells us where the biggest jumps are. We notice that having an accurate face detection mechanism really improves the performance, and similarly, the eyes really help making the prediction more accurate.

Error analysis is also useful when publishing a paper, since it's a convenient way to

Component	Accuracy
Overall system	85%
Preprocess (remove background)	85.1%
Face detection	91%
Eyes segmentation	95%
Nose segmentation	96%
Mouth segmentation	97%
Logistic regression	100%

Table 1: Accuracy when providing the system with the perfect component

analyze the error of an algorithm and explain which parts should be improved.

Ablative analysis

While error analysis tries to explain the difference between current performance and perfect performance, ablative analysis tries to explain the difference between some baseline (much poorer) performance and current performance.

For instance, suppose you have built a good anti-spam classifier by adding lots of clever features to logistic regression

- Spelling correction
- Sender host features
- Email header features
- Email text parser features
- Javascript parser
- Features from embedded images

and your question is: How much did each of these components really help?

In this example, let's say that simple logistic regression without any clever features gets 94% performance, but when adding these clever features, we get 99.9% performance. In ablative analysis, what we do is start from the current level of performance 99.9%, and slowly take away all of these features to see how it affects performance. The results are provided in table 2.

When presenting the results in a paper, ablative analysis really helps analyzing the features that helped decreasing the misclassification rate. Instead of simply giving the loss/error rate of the algorithm, we can provide evidence that some specific features are actually more important than others.

Component	Accuracy
Overall system	99.9%
Spelling correction	99.0%
Sender host features	98.9%
Email header features	98.9%
Email text parser features	95%
Javascript parser	94.5%
Features from images	94.0%

Table 2: Accuracy when removing feature from logistic regression

Analyze your mistakes

Assume you are given a dataset with pictures of animals, and your goal is to identify pictures of cats that you would eventually send to the members of a community of cat lovers. You notice that there are many pictures of dogs in the original dataset, and wonders whether you should build a special algorithm to identify the pictures of dogs and avoid sending dogs pictures to cat lovers or not.

One thing you can do is take a 100 examples from your development set that are misclassified, and count up how many of these 100 mistakes are dogs. If 5% of them are dogs, then even if you come up with a solution to identify your dogs, your error would only go down by 5%, that is your accuracy would go up from 90% to 90.5%. However, if 50 of these 100 errors are dogs, then you could improve your accuracy to reach 95%.

By analyzing your mistakes, you can focus on what's really important. If you notice that 80 out of your 100 mistakes are blurry images, then work hard on classifying correctly these blurry images. If you notice that 70 out of the 100 errors are great cats, then focus on this specific task of identifying great cats.

In brief, do not waste your time improving parts of your algorithm that won't really help decreasing your error rate, and focus on what really matters.